

**Web Programming Using Python and
JavaScript Lab
[SEC035]
Lab Report Submitted to**



**Bachelor of Technology
In
Computer Science Engineering
SEMESTER-IV**

**Submitted by
SHIKHA-2401010063
SECTION-B**

**Course Teacher
Ms. Mansi Kajal**

**School of Engineering & Technology
K. R. MANGALAM UNIVERSITY**

INDEX

S. No.	Experiment Title	Date	Signature
1	Personal Portfolio Website (HTML, CSS, JS)		
2	Company Landing Page – Bright Solution		
3	Interactive Quiz Application		
4	Dynamic To-Do List (JavaScript)		
5	Simple Blog Application (Flask)		
6	Contact Management System (Flask)		
7	E-Commerce Website (HTML, CSS, JS)		

EXPERIMENT–1

AIM- To design and develop a personal portfolio website using HTML, CSS, and JavaScript that displays personal information, skills, projects, services, and contact details in a structured, attractive, and interactive manner.

Technologies Used-

- HTML – For creating the structure of the website
- CSS – For styling, layout, colors, and responsiveness
- JavaScript – For adding interactivity like buttons, show/hide, and navigation effects
- Web Browser – To run and test the website

Explanation-

HTML Structure

- index.html contains the main structure of the portfolio.
- Semantic tags such as <header>, <nav>, <section>, and <footer> are used.
- Separate HTML pages are created for different services like frontend, database, and Java.

CSS Styling

- style.css is linked externally to all HTML files.
- CSS is used for:
 - Layout alignment
 - Background colors and fonts
 - Spacing using margin and padding
 - Styling buttons, cards, and sections

JavaScript Functionality

- script.js is used to:
 - Handle button clicks
 - Add interactive effects
 - Improve navigation experience

HTML CODE:

resume.html > html > body > section#contact.contact.reveal

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Shikha | Portfolio</title>
6   <link rel="stylesheet" href="re.css">
7 </head>
8 <body>
9
10 <!-- NAVBAR -->
11 <header class="navbar">
12   <div class="logo">SJ</div>
13   <nav>
14     <a href="#about">About</a>
15     <a href="#skills">Skills</a>
16     <a href="#projects">Projects</a>
17     <a href="#services">Services</a>
18     <a href="#contact">Contact</a>
19   </nav>
20 </header>
21
22 <!-- HERO / ABOUT -->
23 <section id="about" class="hero reveal">
24   <div class="hero-img">
25     
26   </div>
27
28   <div class="hero-text">
29     <h1>Hey, I'm <span>Shikha</span></h1>
30     <p>
31       B.Tech CSE student exploring core CS concepts and web development.
32       Learning by building, breaking, and improving every day.
33     </p>
34   </div>
35 </section>
```

```
<!-- SKILLS -->
<section id="skills" class="skills reveal">
  <h2>Skills</h2>
  <div class="skill-box">
    <h1 style="font-size: 24px;">Technologies and Frameworks</h1>
    <span>Git</span>
    <span>GitHub</span>
    <span>VS Code</span>
    <span>Netlify</span>
    <h1 style="font-size: 24px; margin-top: 20px;">Programming Languages</h1>
    <span>HTML</span>
    <span>CSS</span>
    <span>JavaScript</span>
    <span>Python</span>
    <span>C++</span>
    <h1 style="font-size: 24px; margin-top: 20px;">Core Concepts</h1>
    <span>Data Structures</span>
    <span>Algorithms</span>
    <span>Database Management Systems</span>
    <h1 style="font-size: 24px; margin-top: 20px;">Soft Skills</h1>
    <span>Team Collaboration</span>
    <span>Time Management</span>
    <span>Active Listening</span>
    <span>Problem Solving</span>
  </div>
</section>
```

```

<!-- PROJECTS -->
<section id="projects" class="projects reveal">
  <h2>Projects</h2>

  <div class="project-grid">
    <div class="project-card" onclick="openProject('project1.html')">
      <h3>InStock - Inventory Management System</h3>
      <p>A web application for managing inventory in retail stores.</p>
    </div>

    <div class="project-card" onclick="openProject('project2.html')">
      <h3>Recipe Haven</h3>
      <p>A web application for managing and sharing recipes.</p>
    </div>
  </div>
</section>

```

```

<body>

<section id="contact" class="contact reveal">
  <h2>Contact Me</h2>

  <div class="contact-links">
    <a href="mailto:jha89719@gmail.com">✉ Email</a>
    <a href="https://linkedin.com" target="_blank">🔗 LinkedIn</a>
  </div>

  <p class="msg-trigger" onclick="openModal()">Send me a message 💬</p>

  <div class="modal" id="msgModal">
    <div class="modal-box">
      <span class="close" onclick="closeModal()">✕</span>

      <h3>How would you like to send your message?</h3>

      <div class="modal-options">
        <button type="button" onclick="showAnonymous()">Send Anonymously</button>
        <button type="button" onclick="showNormal()">Send With Name</button>
      </div>

      <form class="form anon-form">
        <textarea placeholder="Your Message" required></textarea>
        <button type="submit">Send</button>
      </form>

      <form class="form normal-form">
        <input type="text" placeholder="Your Name" required>
        <input type="email" placeholder="Your Email" required>
        <textarea placeholder="Your Message" required></textarea>
        <button type="submit">Send</button>
      </form>
    </div>
  </div>
</div>

```

```

38         <button type="submit">Submit</button>
39     </form>
40 </div>
41 </div>
42 </section>
43
44
45
46 <footer>
47     <p>© 2026 Shikha.</p>
48 </footer>
49
50 <script src="on.js"></script>
51
52 </body>
53 </html>
54

```

CSS CODE:

```

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

html {
    scroll-behavior: smooth;
}

body {
    font-family: 'Segoe UI', sans-serif;
    background: #f8fafc;
    color: #111827;
}

/* NAVBAR */
.navbar {
    position: fixed;
    top: 0;
    width: 100%;
    padding: 15px 60px;
    display: flex;
    justify-content: space-between;
    background: white;
    box-shadow: 0 5px 20px rgba(0,0,0,0.05);
    z-index: 100;
}

.navbar a {
    margin-left: 25px;
    text-decoration: none;
    color: #111827;
}

/* HERO */
.hero {

```

```

/* HERO */
.hero {
  min-height: 100vh;
  padding: 120px 80px;
  display: flex;
  align-items: center;
  gap: 80px;
}

.hero-img {
  flex: 1;
  display: flex;
  justify-content: center;
}

.hero-img img {
  width: 360px;
  height: 460px;
  object-fit: cover;
  border-radius: 24px;

  transform: rotate(-4deg);
  box-shadow:
    0 25px 50px 0 rgba(0, 0, 0, 0.18),
    0 10px 20px 0 rgba(0, 0, 0, 0.12);

  transition: transform 0.4s ease;
}

.hero-img img:hover {
  transform: rotate(0deg) scale(1.05);
}

.hero-img::before {
  content: "";
  position: absolute;
  width: 360px;
  height: 460px;
}

.hero-img::before {
}

.hero-img {
  position: relative;
}

.hero-text {
  flex: 1;
}

.hero-text h1 {
  font-size: 48px;
}

.hero-text span {
  color: #6366f1;
}

.hero-text p {
  margin-top: 15px;
  font-size: 18px;
  max-width: 500px;
}

/* SECTIONS */
section {
  padding: 100px 80px;
}

h2 {
  font-size: 36px;
  margin-bottom: 40px;
}

/* SKILLS */
.skill-box span {
  display: inline-block;
  background: white;
}

```

```
.project-card {
  cursor: pointer;
  transition: 0.3s;
  box-shadow: 0 15px 30px rgba(0,0,0,0.08);
}

.project-card:hover {
  transform: translateY(-8px);
}

/* CONTACT */
.contact-links a {
  margin-right: 20px;
  text-decoration: none;
  font-weight: 500;
}

.msg-line {
  margin: 30px 0 10px;
}

.contact-form input,
.contact-form textarea {
  width: 100%;
  padding: 12px;
  margin-bottom: 15px;
}

.contact-form button {
  padding: 12px 25px;
  background: #6366f1;
  color: white;
  border: none;
  border-radius: 8px;
  cursor: pointer;
}
```



```

.anon {
  gap: 8px;
  margin-bottom: 15px;
}

.msg-trigger {
  margin-top: 30px;
  font-size: 18px;
  color: #6366f1;
  cursor: pointer;
}

/* MODAL BACKGROUND */
.modal {
  position: fixed;
  inset: 0;
  background: rgba(0,0,0,0.4);
  display: none;
  align-items: center;
  justify-content: center;
  z-index: 999;
}

/* MODAL BOX */
.modal-box {
  background: white;
  padding: 30px;
  width: 400px;
  border-radius: 16px;
  position: relative;
  animation: scaleIn 0.3s ease;
}

@keyframes scaleIn {
  from { transform: scale(0.9); opacity: 0; }
  to { transform: scale(1); opacity: 1; }
}

```

JAVASCRIPT CODE:

```

document.addEventListener('DOMContentLoaded', () => {

  // ----- PROJECT OPEN -----
  window.openProject = function (page) {
    window.location.href = page;
  };

  // ----- SCROLL REVEAL -----
  const reveals = document.querySelectorAll('.reveal');

  window.addEventListener('scroll', () => {
    reveals.forEach(section => {
      const top = section.getBoundingClientRect().top;
      if (top < window.innerHeight - 100) {
        section.classList.add('active');
      }
    });
  });

  // ----- CLICK HIGHLIGHT -----
  const sections = document.querySelectorAll('.reveal');

  sections.forEach(section => {
    section.addEventListener('click', () => {

      // remove active-click from all
      sections.forEach(sec => sec.classList.remove('active-click'));

      // add to clicked one
      section.classList.add('active-click');
    });
  });

  // ----- MODAL -----
  const modal = document.getElementById("msgModal");
  const anonForm = document.querySelector(".anon-form");
  const normalForm = document.querySelector(".normal-form");

  window.openModal = function () {
    modal.style.display = "flex";
  };

  window.closeModal = function () {
    modal.style.display = "none";
    anonForm.style.display = "none";
    normalForm.style.display = "none";
  };

  window.showAnonymous = function () {
    anonForm.style.display = "block";
    normalForm.style.display = "none";
  };

  window.showNormal = function () {
    normalForm.style.display = "block";
    anonForm.style.display = "none";
  };

});

// ----- CLICK GLOW FOR CARDS -----
const cards = document.querySelectorAll('.project-card, .service-card');

cards.forEach(card => {
  card.addEventListener('click', () => {

    cards.forEach(c => c.classList.remove('active-click'));

    card.classList.add('active-click');
  });
});

```

OUTPUT:



Hey, I'm Shikha

B.Tech CSE student exploring core CS concepts and web development. Learning by building, breaking, and improving every day.

Skills

Technologies and Frameworks

Git GitHub VS Code Netlify

Programming Languages

HTML CSS JavaScript Python C++

Core Concepts

Data Structures Algorithms Database Management Systems

Soft Skills

Team Collaboration Time Management Active Listening Problem Solving

Projects

InStock - Inventory Management System

A web application for managing inventory in retail stores.

Recipe Haven

A web application for managing and sharing recipes.

Contact Me

[Email](#) [LinkedIn](#)

[Send me a message](#)

RESULT: A fully functional, professional-looking personal portfolio website was successfully created using HTML, CSS, and JavaScript.

EXPERIMENT-2

AIM- To design and develop a professional company landing page named Bright Solution using HTML and CSS, displaying company information, services, testimonials, and contact details in a well-structured and visually appealing manner.

Technologies Used- HTML – To create the structure of the landing page - CSS – To design layout, colors, fonts, and overall appearance - Web Browser – To test and run the website

Explanation-

HTML Structure

- index.html contains the main structure of the landing page.
- Semantic tags like <header>, <section>, and <footer> are used.
- Sections included:
 - Header / Hero Section
 - About / Introduction
 - Services
 - Testimonials
 - Contact Information
- Separate HTML pages are created for each service (Web Development, Database, IT Consulting).

CSS Styling

- style.css is linked externally to all pages.
- CSS is used to:
 - Style layout and spacing
 - Apply background colors and fonts
 - Align sections and create service cards
 - Improve readability and visual appeal

HTML CODE:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Bright Solution | Home</title>
  <link rel="stylesheet" href="company.css">
</head>
<body>

<header>
  <div class="container">
    <h1>Bright Solution</h1>
    <p>Your Partner in Digital Growth</p>
    <a href="#contact" class="btn">Get in Touch</a>
  </div>
</header>

<nav>
  <ul>
    <li><a href="index.html">Home</a></li>
    <li><a href="#about">About</a></li>
    <li><a href="#services">Services</a></li>
    <li><a href="#testimonials">Testimonials</a></li>
    <li><a href="#contact">Contact</a></li>
  </ul>
</nav>

<section id="about" class="section">
  <div class="container">
    <h2>About Us</h2>
    <p>Bright Solution is a professional IT company providing innovative technology services to businesses worldwide. We specialize in modern web development, database solutions, and IT consulting to help organizations grow efficiently.</p>
  </div>
</section>

<section id="services" class="section light">
  <div class="container">
    <h2>Our Services</h2>
    <div class="card-container">
      <div class="card">
        <h3>Web Development</h3>
        <p>Modern and responsive website development using latest technology.</p>
        <a href="company2.html" class="btn-small">Read More</a>
      </div>
      <div class="card">
        <h3>Database Solutions</h3>
        <p>Secure and scalable database design and management services.</p>
        <a href="company3.html" class="btn-small">Read More</a>
      </div>
      <div class="card">
        <h3>IT Consulting</h3>
        <p>Strategic IT consulting to optimize your business processes.</p>
        <a href="company4.html" class="btn-small">Read More</a>
      </div>
    </div>
  </div>
</section>

<section id="testimonials" class="section">
  <div class="container">
    <h2>What Our Clients Say</h2>
    <div class="testimonial">
      <p>"Bright Solution transformed our online presence completely!"</p>
      <h4>- Rahul Mehta</h4>
    </div>
  </div>

```

```

        <div class="testimonial">
            <p>"Professional team and excellent support. Highly recommended."</p>
            <h4>- Sneha Kapoor</h4>
        </div>
    </div>
</section>

<section id="contact" class="section light">
    <div class="container">
        <h2>Contact Us</h2>
        <p>Email: info@brightsolution.com</p>
        <p>Phone: +91 9876543210</p>
        <p>Address: New Delhi, India</p>
    </div>
</section>

<footer>
    <p>© 2026 Bright Solution. All Rights Reserved.</p>
</footer>

</body>
</html>

```

CSS CODE:

```

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: Arial, sans-serif;
}

body {
    line-height: 1.6;
}

.container {
    width: 80%;
    margin: auto;
}

header {
    background: linear-gradient(to right, #007bff, #00c6ff);
    color: white;
    text-align: center;
    padding: 80px 20px;
}

header h1 {
    font-size: 48px;
}

header p {
    font-size: 20px;
    margin: 20px 0;
}

nav {
    background: #222;
    padding: 15px 0;
}

```

```

nav ul li a {
  text-decoration: none;
  font-weight: bold;
}

nav ul li a:hover {
  color: #00c6ff;
}

.section {
  padding: 60px 0;
  text-align: center;
}

.section h2 {
  margin-bottom: 20px;
  font-size: 32px;
}

.light {
  background: #f4f4f4;
}

.card-container {
  display: flex;
  justify-content: space-between;
  margin-top: 40px;
  flex-wrap: wrap;
}

.card {
  background: white;
  padding: 20px;
  width: 30%;
  box-shadow: 0 4px 10px rgba(0,0,0,0.1);
  border-radius: 8px;
  margin-bottom: 20px;
}

```

```

.btn, .btn-small {
  display: inline-block;
  background: #007bff;
  color: white;
  padding: 10px 20px;
  text-decoration: none;
  border-radius: 5px;
  margin-top: 15px;
}

.btn:hover, .btn-small:hover {
  background: #0056b3;
}

.testimonial {
  margin: 20px 0;
  font-style: italic;
}

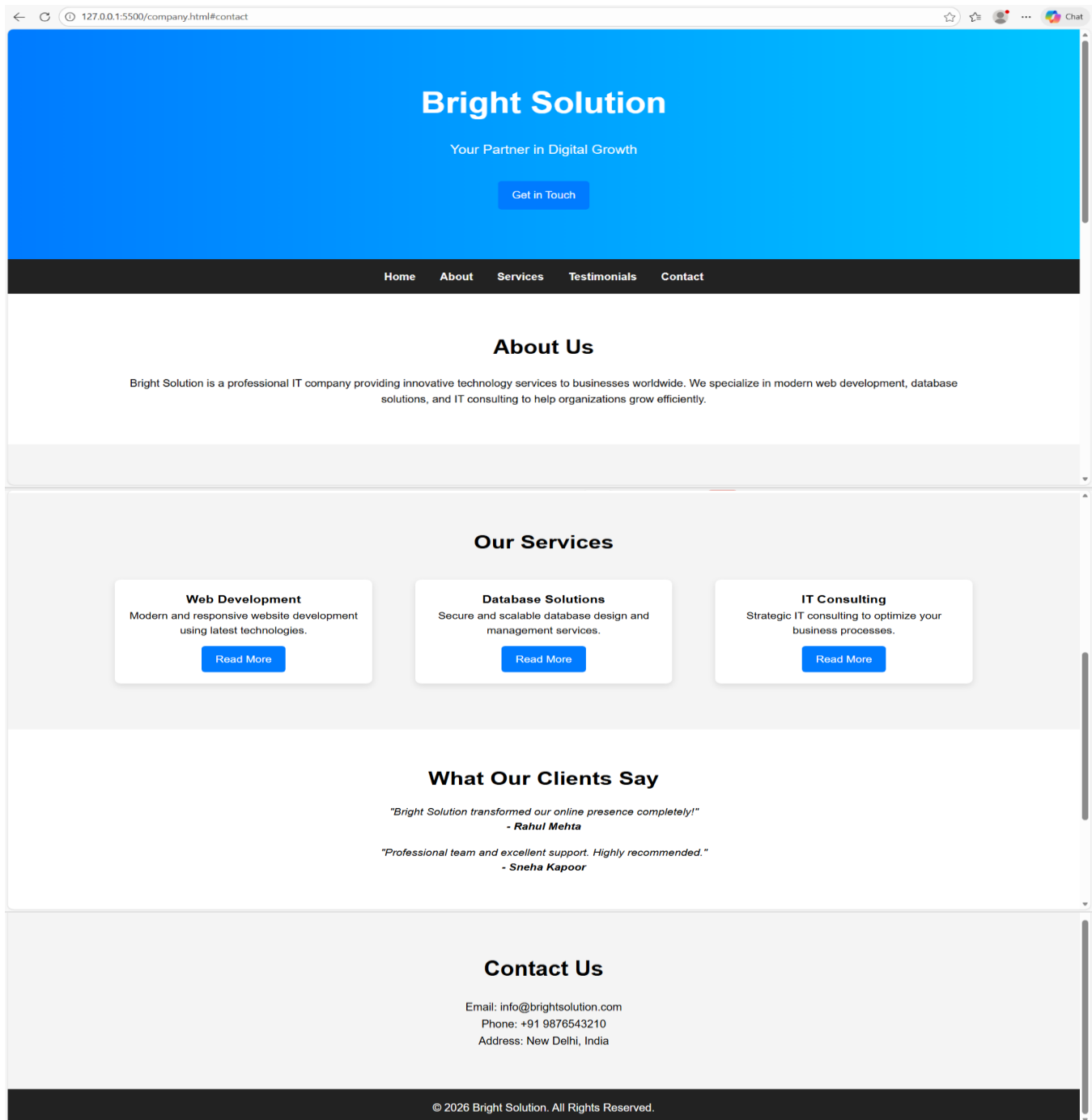
footer {
  background: #222;
  color: white;
  text-align: center;
  padding: 15px 0;
}

/* Responsive */
@media(max-width: 768px) {
  .card {
    width: 100%;
  }

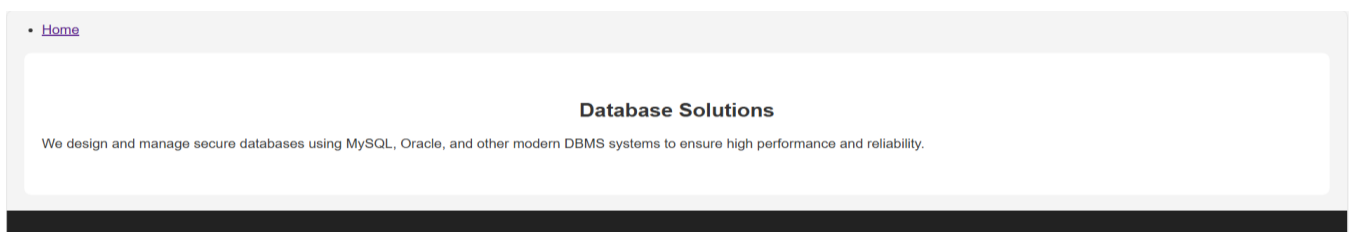
  header h1 {
    font-size: 32px;
  }
}

```

OUTPUT:



SERVICE SECTION



RESULT: A complete and visually appealing company landing page for Bright Solution was successfully developed using HTML and CSS.

EXPERIMENT–3

AIM- To design and develop an interactive quiz application using HTML, CSS, and JavaScript that displays questions, records user responses, tracks the score, shows results, and allows the quiz to be restarted.

Technologies Used:

- HTML – For creating the structure of the quiz interface
- CSS – For styling the quiz layout and improving user experience
- JavaScript – For quiz logic, interactivity, timer, and result calculation
- Web Browser – To run and test the application

Explanation

HTML Structure

HTML is used to create the layout of the quiz using semantic elements such as <header>, <section>, and <button>.

It displays:

- Quiz title and introduction
- Question text
- Multiple-choice options (radio buttons)
- Next button
- Result section and feedback form

CSS Styling

CSS is applied to:

- Improve readability of questions and options
- Align sections and buttons properly
- Use colors, spacing, and borders for better user experience
- Make the quiz interface clean and professional

JavaScript Functionality

JavaScript is used to:

- Store questions and answers in an array of objects
- Capture user-selected answers
- Move between questions using button events

- Track and update the score
- Display final results and feedback messages
- Implement a countdown timer for each question
- Allow the quiz to restart

HTML CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Smart Quiz App</title>
  <link rel="stylesheet" href="quiz.css">
</head>
<body>

<header>
  <h1>Smart Quiz Challenge</h1>
  <p>Test Your Knowledge in 5 Quick Questions!</p>
</header>

<section class="quiz-container">

  <div id="quiz-box">
    <div id="timer">Time Left: <span id="time">15</span>s</div>
    <div id="progress">Question 1 of 5</div>

    <h2 id="question">Question text</h2>

    <div id="options"></div>

    <button id="next-btn">Next</button>
  </div>

  <div id="result-box" class="hidden">
    <h2>Your Result</h2>
    <p id="score-text"></p>
    <p id="feedback"></p>
    <button id="restart-btn">Restart Quiz</button>
  </div>

</section>

<script src="quiz.js"></script>
```

CSS CODE:

```

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: Arial, sans-serif;
}

body {
  background: linear-gradient(to right, #4e73df, #1cc88a);
  display: flex;
  flex-direction: column;
  align-items: center;
  min-height: 100vh;
  color: #333;
}

header {
  text-align: center;
  padding: 30px;
  color: white;
}

.quiz-container {
  background: white;
  width: 400px;
  padding: 25px;
  border-radius: 10px;
  box-shadow: 0 5px 15px rgba(0,0,0,0.2);
}

#timer {
  font-weight: bold;
  margin-bottom: 10px;
  color: #e74a3b;
}

#progress {

```

```

#question {
  margin-bottom: 15px;
}

#options label {
  display: block;
  margin-bottom: 10px;
  padding: 8px;
  border: 1px solid #ddd;
  border-radius: 5px;
  cursor: pointer;
  transition: 0.3s;
}

#options label:hover {
  background-color: #f8f9fc;
}

button {
  margin-top: 15px;
  padding: 10px;
  width: 100%;
  border: none;
  background: #4e73df;
  color: white;
  border-radius: 5px;
  cursor: pointer;
  font-size: 16px;
}

button:hover {
  background: #2e59d9;
}

.hidden {
  display: none;
}

```

JAVASCRIPT CODE:

```
const questions = [
  {
    question: "Which language is used for styling web pages?",
    options: ["HTML", "Python", "CSS", "C++"],
    answer: "CSS"
  },
  {
    question: "Which company developed JavaScript?",
    options: ["Microsoft", "Netscape", "Google", "Oracle"],
    answer: "Netscape"
  },
  {
    question: "What does SQL stand for?",
    options: [
      "Structured Query Language",
      "Simple Query Language",
      "System Query Logic",
      "Server Question Language"
    ],
    answer: "Structured Query Language"
  },
  {
    question: "Which HTML tag is used to link JavaScript?",
    options: ["<js>", "<javascript>", "<script>", "<link>"],
    answer: "<script>"
  },
  {
    question: "Which property is used to change text color in CSS?",
    options: ["font-color", "text-color", "color", "background-color"],
    answer: "color"
  }
];

let currentQuestion = 0;
let score = 0;
let timer;
let timeLeft = 15;

const questionEl = document.getElementById("question");
const optionsEl = document.getElementById("options");
const nextBtn = document.getElementById("next-btn");
const timeEl = document.getElementById("time");
const progressEl = document.getElementById("progress");
const resultBox = document.getElementById("result-box");
const quizBox = document.getElementById("quiz-box");
const scoreText = document.getElementById("score-text");
const feedback = document.getElementById("feedback");
const restartBtn = document.getElementById("restart-btn");

function startTimer() {
  timeLeft = 15;
  timeEl.textContent = timeLeft;

  timer = setInterval(() => {
    timeLeft--;
    timeEl.textContent = timeLeft;

    if (timeLeft <= 0) {
      clearInterval(timer);
      nextQuestion();
    }
  }, 1000);
}

function loadQuestion() {
  clearInterval(timer);
  startTimer();

  const q = questions[currentQuestion];
  questionEl.textContent = q.question;
  optionsEl.innerHTML = "";

  progressEl.textContent = `Question ${currentQuestion + 1} of ${questions.length}`;
}
```

```

function nextQuestion() {
    currentQuestion++;

    if (currentQuestion < questions.length) {
        loadQuestion();
    } else {
        showResult();
    }
}

function showResult() {
    quizBox.classList.add("hidden");
    resultBox.classList.remove("hidden");

    scoreText.textContent = `You scored ${score} out of ${questions.length}`;

    if (score === questions.length) {
        feedback.textContent = "Outstanding Performance! 🏆";
    } else if (score >= 3) {
        feedback.textContent = "Good Job! Keep Practicing 👍";
    } else {
        feedback.textContent = "Keep Learning! You Can Improve 📈";
    }
}

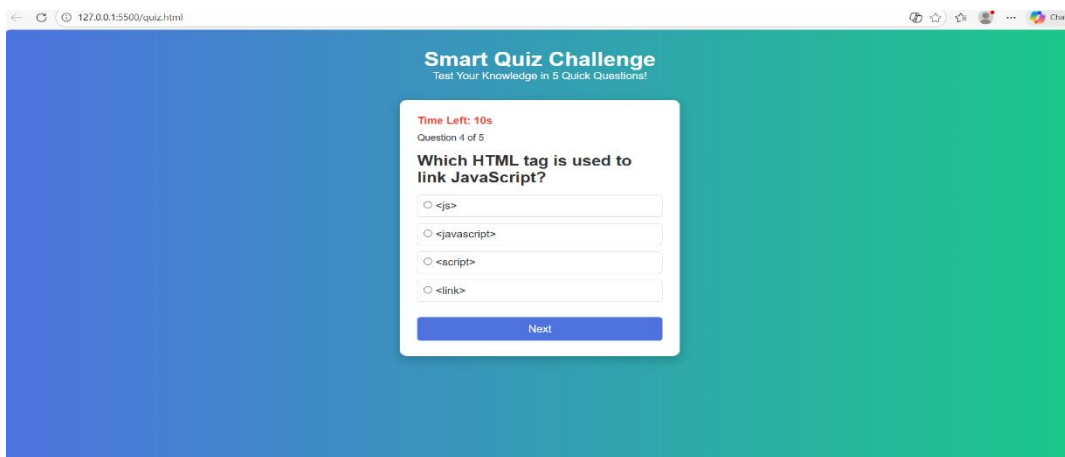
function restartQuiz() {
    currentQuestion = 0;
    score = 0;
    resultBox.classList.add("hidden");
    quizBox.classList.remove("hidden");
    loadQuestion();
}

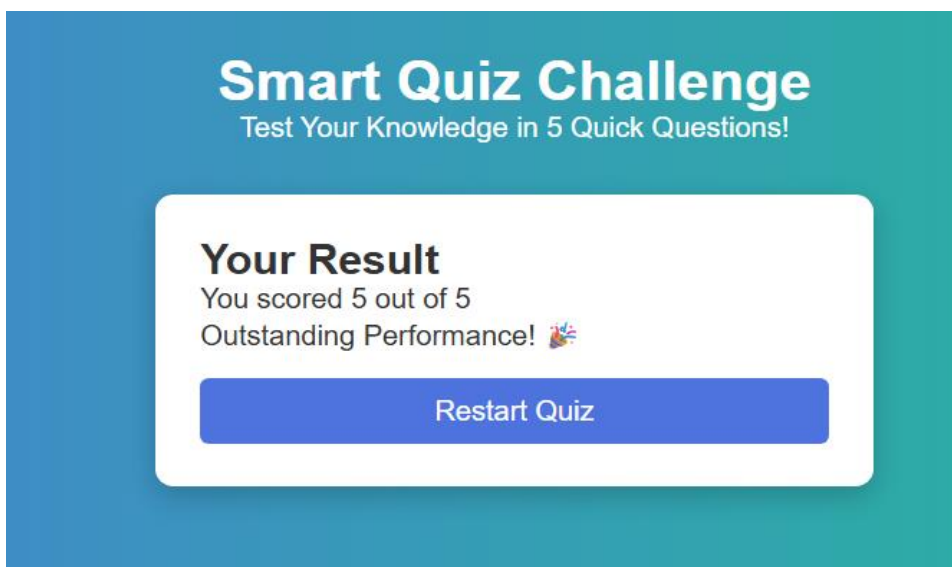
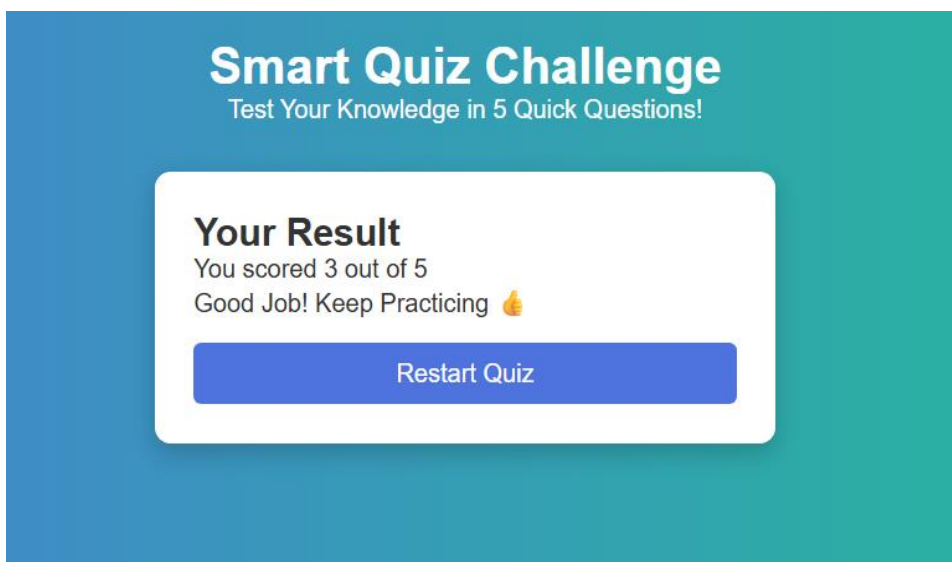
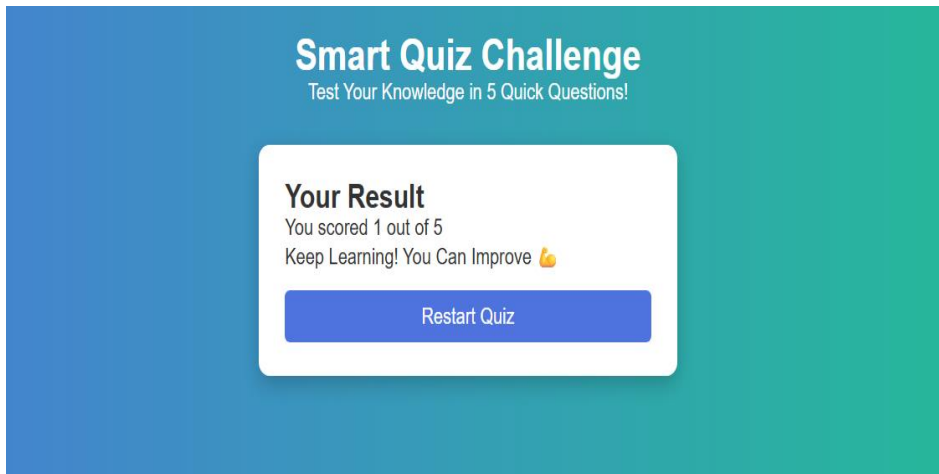
nextBtn.addEventListener("click", nextQuestion);
restartBtn.addEventListener("click", restartQuiz);

loadQuestion();

```

OUTPUT:





RESULT: An interactive quiz application was successfully created using HTML, CSS, and JavaScript. The application is user-friendly, visually appealing, and meets all the experiment requirements including scoring, result display, feedback, and restart functionality.

EXPERIMENT-4

AIM: To design and develop a dynamic To-Do List application using HTML, CSS, and JavaScript that allows users to add, edit, delete, and manage tasks efficiently.

Technologies Used

- **HTML** – To create structure of the application
- **CSS** – To style layout and improve UI
- **JavaScript** – To implement dynamic functionality
- **Web Browser** – To run and test the application

Explanation

HTML Structure

- index.html contains the main layout of the application
- Includes:
 - Input field for tasks
 - Add Task button
 - Task list display ()
 - Task statistics section

CSS Styling

- style.css is used for:
 - Designing input and buttons
 - Styling task list items
 - Highlighting completed tasks (strikethrough)
 - Improving spacing and layout

JavaScript Functionality

- Tasks stored in an array
- Functions implemented:
 - Add Task
 - Display Tasks
 - Edit Task
 - Delete Task
 - Mark task as completed

- Prevents empty tasks
- Updates UI dynamically using DOM manipulation

CODE

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>To-Do List</title>
  <link rel="stylesheet" href="style.css">
</head>

<body>

<div class="container">
  <h1> 📅 To-Do List</h1>
  <p>Manage your daily tasks easily</p>

  <div class="input-section">
    <input type="text" id="taskInput" placeholder="Enter a task">
    <button onclick="addTask()">Add Task</button>
  </div>

  <ul id="taskList"></ul>

  <p id="stats"></p>
</div>

<script src="script.js"></script>
</body>
</html>
```

Script.js


```

// 🔄 TOGGLE COMPLETE
let tasks = [];

// ➕ ADD TASK
function addTask() {
  const input = document.getElementById("taskInput");
  const text = input.value.trim();

  if (text === "") {
    alert("Task cannot be empty!");
    return;
  }

  tasks.push({
    text: text,
    completed: false
  });

  input.value = "";
  renderTasks();
}

// 📋 RENDER TASKS
function renderTasks() {
  const list = document.getElementById("taskList");
  list.innerHTML = "";

  let completedCount = 0;

  tasks.forEach((task, index) => {
    if (task.completed) completedCount++;

    const li = document.createElement("li");
    li.className = task.completed ? "completed" : "";

    li.innerHTML = `
      <div class="task-left">
        <input type="checkbox"
    `;

    tasks.forEach((task, index) => {
      li.innerHTML += `
        <span>${task.text}</span>
      </div>

      <div class="task-buttons">
        <button onclick="editTask(${index})">Edit</button>
        <button class="delete" onclick="deleteTask(${index})">Delete</button>
      </div>
    `;

    list.appendChild(li);
  });

  document.getElementById("stats").innerText =
    `Completed: ${completedCount} | Pending: ${tasks.length - completedCount}`;
}

// ✖ DELETE TASK
function deleteTask(index) {
  tasks.splice(index, 1);
  renderTasks();
}

// ✏ EDIT TASK
function editTask(index) {
  let newText = prompt("Edit your task:", tasks[index].text);

  if (newText !== null && newText.trim() !== "") {
    tasks[index].text = newText.trim();
    renderTasks();
  }
}

// ✅ TOGGLE COMPLETE (checkbox)
function toggleComplete(index) {

```

Style.css

```
body {  
  font-family: Arial;  
  background: #eef2f7;  
  margin: 0;  
}  
  
.container {  
  width: 400px;  
  margin: 50px auto;  
  background: white;  
  padding: 20px;  
  border-radius: 10px;  
  box-shadow: 0 5px 15px rgba(0,0,0,0.1);  
}  
  
h1 {  
  text-align: center;  
}  
  
.input-section {  
  display: flex;  
  gap: 10px;  
}  
  
input {  
  flex: 1;  
  padding: 8px;  
}  
  
button {  
  padding: 8px;  
  background: #3498db;  
  color: white;  
  border: none;  
  cursor: pointer;  
}
```

```

button:hover {
  background: #2980b9;
}

li {
  list-style: none;
  padding: 10px;
  margin-top: 10px;
  background: #f4f4f4;
  display: flex;
  justify-content: space-between;
  align-items: center;
  border-radius: 5px;
}

.completed {
  text-decoration: line-through;
  color: gray;
}

.task-buttons button {
  margin-left: 5px;
  background: #2ecc71;
}

.task-buttons .delete {
  background: red;
}

.completed span {
  text-decoration: line-through;
  color: gray;
}

li {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

```

OUTPUT



To-Do List

Manage your daily tasks easily

Add Task

Completed: 0 | Pending: 0



To-Do List

Manage your daily tasks easily

Add Task

☐ Study

EditDelete

☐ Meeting

EditDelete

Completed: 0 | Pending: 2



To-Do List

Manage your daily tasks easily

Add Task☒ StudyEditDelete☐ MeetingEditDelete

Completed: 1 | Pending: 1

RESULT

A dynamic To-Do List application was successfully developed using HTML, CSS, and JavaScript with full functionality including adding, editing, deleting, and tracking task completion.

EXPERIMENT-5

AIM: To design and develop a simple Blog Application using Flask (Python) that allows users to create, read, update, and delete blog posts through a web interface.

Technologies Used

- **Python (Flask)** – Backend development and routing
- **HTML** – Structure of web pages
- **CSS** – Styling and layout
- **Web Browser** – To run and test the application

Explanation

Project Structure

- app.py – Main Flask application handling routes and logic
- templates/ – Contains HTML files:
 - base.html – Common layout
 - index.html – Displays all blog posts
 - create.html – Form to create a post
 - edit.html – Form to edit a post
- static/ – Contains CSS for styling

Flask Functionality

- Flask is used to manage routing and backend logic
- Blog posts are stored in a list (no database used)

Routes Implemented

- / → Displays all blog posts
- /create → Create a new blog post
- /edit/<id> → Update an existing post
- /delete/<id> → Delete a post

Features Implemented

- Create new blog posts using a form
- Display all posts on the homepage

- Edit existing posts with pre-filled data
- Delete posts dynamically
- Navigation between pages

CODE

App.py

```
app.py > ...
1 | from flask import Flask, render_template, request, redirect, url_for
2
3 | app = Flask(__name__)
4
5 | posts = []
6 | post_id_counter = 1
7
8
9 | @app.route('/')
10 | def index():
11 |     return render_template('index.html', posts=posts)
12
13
14 | @app.route('/create', methods=['GET', 'POST'])
15 | def create():
16 |     global post_id_counter
17
18 |     if request.method == 'POST':
19 |         title = request.form.get('title')
20 |         content = request.form.get('content')
21
22 |         if title and content:
23 |             posts.append({
24 |                 'id': post_id_counter,
25 |                 'title': title,
26 |                 'content': content
27 |             })
28 |             post_id_counter += 1
29
30 |         return redirect(url_for('index'))
31
32 |     return render_template('create.html')
33
34
35 | @app.route('/edit/<int:id>', methods=['GET', 'POST'])
36 | def edit(id):
37 |     post = next((p for p in posts if p['id'] == id), None)
```

```

post = next((p for p in posts if p['id'] == id), None)

if not post:
    return redirect(url_for('index'))

if request.method == 'POST':
    post['title'] = request.form.get('title')
    post['content'] = request.form.get('content')
    return redirect(url_for('index'))

return render_template('edit.html', post=post)

@app.route('/delete/<int:id>')
def delete(id):
    global posts
    posts = [p for p in posts if p['id'] != id]
    return redirect(url_for('index'))

if __name__ == '__main__':
    app.run(debug=True)

```

Base.html

```

nplates > base.html > html
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Modern Blog</title>
5      <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
6  </head>
7  <body>
8
9  <nav class="navbar">
10     <div class="logo">💎 MyBlog</div>
11     <div class="nav-links">
12         <a href="/">Home</a>
13         <a href="/create" class="btn">+ New Post</a>
14     </div>
15 </nav>
16
17 <div class="container">
18     {% block content %}{% endblock %}
19 </div>
20
21 </body>
22 </html>

```

Create.html


```
plates > <> create.html > ...
{% extends 'base.html' %}

{% block content %}

<div class="form-box">
<h2>Create Post</h2>

<form method="POST">
  <input type="text" name="title" placeholder="Post Title" required>
  <textarea name="content" placeholder="Write your content..." required></textarea>
  <button type="submit">Publish</button>
</form>

</div>

{% endblock %}
```

Edit.html

```
lates > <> edit.html > ...
{% extends 'base.html' %}

{% block content %}

<div class="form-box">
<h2>Edit Post</h2>

<form method="POST">
  <input type="text" name="title" value="{{ post.title }}" required>
  <textarea name="content" required>{{ post.content }}</textarea>
  <button type="submit">Update</button>
</form>

</div>

{% endblock %}
```

Index.html

```
plates > <> index.html > <> div.grid > <> div.card
{% extends 'base.html' %}

{% block content %}

<h1><img alt="Python logo" data-bbox="195 668 215 680"/> Latest Posts</h1>

{% if posts %}
<div class="grid">
  {% for post in posts %}
    <div class="card">
      <h2>{{ post.title }}</h2>
      <p>{{ post.content[:120] }}...</p>

      <div class="actions">
        <a href="/edit/{{ post.id }}" class="edit">Edit</a>
        <a href="/delete/{{ post.id }}"
          class="delete"
          onclick="return confirm('Delete this post?')">Delete</a>
      </div>
    </div>
  {% endfor %}
</div>
{% else %}
<div class="empty">
  <h2>No posts yet 😞</h2>
  <p>Create your first blog post!</p>
</div>
{% endif %}

{% endblock %}
```

Style.css

```
/* style.css / .empty
body {
  font-family: 'Segoe UI';
  background: #f5f7fb;
  margin: 0;
}

/* NAVBAR */
.navbar {
  display: flex;
  justify-content: space-between;
  padding: 15px 30px;
  background: #111;
  color: white;
}

.nav-links a {
  margin-left: 15px;
  text-decoration: none;
  color: white;
}

.btn {
  background: #4CAF50;
  padding: 6px 12px;
  border-radius: 5px;
}

/* CONTAINER */
.container {
  padding: 30px;
}

/* GRID */
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
  gap: 20px;
}

.edit {
  margin-right: 10px;
}

.delete {
  color: red;
}

/* FORM */
.form-box {
  max-width: 500px;
  margin: auto;
  background: white;
  padding: 25px;
  border-radius: 10px;
}

form input, form textarea {
  width: 100%;
  padding: 10px;
  margin: 10px 0;
}

button {
  width: 100%;
  padding: 10px;
  background: black;
  color: white;
  border: none;
}

/* EMPTY STATE */
.empty {
  text-align: center;
  margin-top: 50px;
}
```



Latest Posts

Birthday!!

Happy Birthday! 🎉 Today isn't just another day—it's a celebration of you and the light you bring into the lives of every...

[Edit](#) [Delete](#)

Create Post

Birthday!!

Happy Birthday! 🎉 Today isn't just another day—it's a celebration of you and the light you bring into the lives of everyone around you. You have a way of making people feel seen, valued, and loved, and I'm so grateful to have you in my life. May this year bring you endless moments of joy, exciting adventures, and the courage to chase every dream in your heart. You deserve all the happiness in the world, and I hope today is just the beginning of something truly beautiful. ❤️|

Publish

Edit Post

Birthday!!

Happy Birthday! 🎉 Today isn't just another day—it's a celebration of you and the light you bring into the lives of everyone around you. You have a way of making people feel seen.

Update

RESULT: A simple blog application was successfully developed using Flask, implementing full CRUD operations with a clean interface and proper navigation.

EXPERIMENT-6

AIM: To design and develop a Contact Management System using Flask (Python), allowing users to perform CRUD operations (Create, Read, Update, Delete) on contact data in a structured and user-friendly interface.

Technologies Used

- **Python (Flask)** – Backend development
- **HTML** – Structure of web pages
- **CSS** – Styling and layout
- **Web Browser** – To run and test application

Explanation

Application Structure

- app.py handles routing and backend logic
- templates/ folder contains HTML pages
- static/ folder contains CSS styling

Features Implemented

- Home page to display contacts
- Add new contact form
- Edit existing contact
- Delete contact
- Search functionality

Flask Functionality

- Routes are used to handle different operations:
 - / → Display contacts
 - /add → Add new contact
 - /edit/<id> → Update contact
 - /delete/<id> → Delete contact
- Contacts are stored in a list (no database used)

CODE

Base.html

```
templates > base.html > html
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Contact Manager</title>
5      <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
6  </head>
7  <body>
8      <div class="container">
9          <h1>📞 Contact Management System</h1>
10         {% block content %}{% endblock %}
11     </div>
12 </body>
13 </html>
```

Add_contact.html

```
{% extends 'base.html' %}

{% block content %}

<h2>Add Contact</h2>

<form method="POST">
    <input type="text" name="name" placeholder="Name">
    <input type="text" name="phone" placeholder="Phone">
    <input type="email" name="email" placeholder="Email">

    <button class="btn">Add</button>
</form>

{% endblock %}
```

Edit_contact.html

```
{% extends 'base.html' %}

{% block content %}

<h2>Edit Contact</h2>

<form method="POST">
    <input type="text" name="name" value="{{ contact.name }}">
    <input type="text" name="phone" value="{{ contact.phone }}">
    <input type="email" name="email" value="{{ contact.email }}">

    <button class="btn">Update</button>
</form>

{% endblock %}
```

Index.html

```
plates > index.html > table
1  {% extends 'base.html' %}
2
3  {% block content %}
4
5  <a href="/add" class="btn">+ Add Contact</a>
6
7  <form method="GET">
8    <input type="text" name="search" placeholder="Search by name or phone">
9  </form>
10
11  <table>
12    <tr>
13      <th>Name</th>
14      <th>Phone</th>
15      <th>Email</th>
16      <th>Actions</th>
17    </tr>
18
19    {% for c in contacts %}
20    <tr>
21      <td>{{ c.name }}</td>
22      <td>{{ c.phone }}</td>
23      <td>{{ c.email }}</td>
24      <td>
25        <a href="/edit/{{ c.id }}" class="btn btn-edit">Edit</a>
26        <a href="/delete/{{ c.id }}" class="btn btn-danger">Delete</a>
27      </td>
28    </tr>
29    {% endfor %}
30  </table>
31
32  {% endblock %}
```

Style.css

```
style.css > input
body {
  font-family: Arial;
  background: #f5f6fa;
  margin: 0;
}

.container {
  width: 80%;
  margin: auto;
  padding: 20px;
}

h1 {
  text-align: center;
}

.btn {
  padding: 8px 12px;
  background: #3498db;
  color: white;
  text-decoration: none;
  border-radius: 5px;
}

.btn-danger {
  background: red;
}

.btn-edit {
  background: orange;
}

table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 20px;
}
```

App.py

```
from flask import Flask, render_template, request, redirect, url_for

app = Flask(__name__)

# In-memory storage (no DB)
contacts = []
contact_id_counter = 1

# 🏠 Home Route (Read)
@app.route('/')
def index():
    search_query = request.args.get('search', '').lower()

    if search_query:
        filtered_contacts = [
            c for c in contacts
            if search_query in c['name'].lower() or search_query in c['phone']
        ]
    else:
        filtered_contacts = contacts

    return render_template('index.html', contacts=filtered_contacts)

# ➕ Add Contact (Create)
@app.route('/add', methods=['GET', 'POST'])
def add_contact():
    global contact_id_counter

    if request.method == 'POST':
        name = request.form['name']
        phone = request.form['phone']
        email = request.form['email']
```

```
pp.py > edit_contact
def add_contact():

    return redirect(url_for('index'))

    return render_template('add_contact.html')

# ✏ Edit Contact (Update)
@app.route('/edit/<int:id>', methods=['GET', 'POST'])
def edit_contact(id):
    contact = next((c for c in contacts if c['id'] == id), None)

    if not contact:
        return "Contact not found!"

    if request.method == 'POST':
        contact['name'] = request.form['name']
        contact['phone'] = request.form['phone']
        contact['email'] = request.form['email']

    return redirect(url_for('index'))

    return render_template('edit_contact.html', contact=contact)

# ✖ Delete Contact
@app.route('/delete/<int:id>')
def delete_contact(id):
    global contacts
    contacts = [c for c in contacts if c['id'] != id]

    return redirect(url_for('index'))

if __name__ == '__main__':
    app.run(debug=True)
```


OUTPUT

 **Contact Management System**

[+ Add Contact](#)

Name	Phone	Email	Actions
------	-------	-------	---------

 **Contact Management System**

Add Contact

[Add](#)

Name	Phone	Email	Actions
Shikha	90908290120	jhas89719@gmail.com	Edit Delete

RESULT

A fully functional Contact Management System was successfully developed using Flask, implementing CRUD operations with a clean and user-friendly interface.

EXPERIMENT-7

AIM: To design and develop a simple E-commerce website using HTML, CSS, and JavaScript with product listing, shopping cart functionality, and checkout simulation.

Technologies Used

- **HTML** – Structure of web pages
- **CSS** – Styling and layout
- **JavaScript** – Dynamic functionality and cart operations
- **Web Browser** – To run and test

Explanation

Website Structure

- index.html → Product listing page
- cart.html → Cart page
- checkout.html → Checkout page
- style.css → Styling
- script.js → Cart logic

Features Implemented

- Product listing with name, price, description
- Add to cart functionality
- Dynamic cart update
- Quantity increase/decrease
- Remove items from cart
- Total price calculation
- Checkout form with confirmation

JavaScript Functionality

- Products stored in array

- Cart stored using localStorage
- DOM manipulation used for dynamic updates
- Functions:
 - Add to cart
 - Update cart
 - Remove items
 - Calculate total

CODE

Car.html

```

art.html > html > body > nav > h2
<!DOCTYPE html>
<html>
<head>
  <title>Cart</title>
  <link rel="stylesheet" href="css/style.css">
</head>

<body>

<nav>
  <h2>🛒 ShopEasy</h2>
  <div>
    <a href="index.html">Home</a>
    <a href="checkout.html">Checkout</a>
  </div>
</nav>

<h1 class="title">Your Cart</h1>

<div class="cart-container">
  <div id="cart-items"></div>
  <h2>Total: ₹<span id="total">0</span></h2>
</div>

<script src="js/script.js"></script>
</body>
</html>

```

Checkout.html

```

Checkout.html > html > body > nav > a
<!DOCTYPE html>
<html>
<head>
  <title>Checkout</title>
  <link rel="stylesheet" href="css/style.css">
</head>

<body>

<nav>
  <h2>🛒 ShopEasy</h2>
  <a href="index.html">Home</a>
</nav>

<h1 class="title">Checkout</h1>

<div class="checkout-box">
  <input type="text" placeholder="Full Name" id="name">
  <input type="text" placeholder="Address" id="address">
  <button onclick="placeOrder()">Place Order</button>
</div>

<script src="js/script.js"></script>
</body>
</html>

```

Index.html

```

7 <!DOCTYPE html>
8 <html>
9 <head>
10   <title>ShopEasy</title>
11   <link rel="stylesheet" href="css/style.css">
12 </head>
13
14 <body>
15
16 <nav>
17   <h2>🛒 ShopEasy</h2>
18   <div>
19     <a href="index.html">Home</a>
20     <a href="cart.html">Cart (<span id="cart-count">0</span></a>
21   </div>
22 </nav>
23
24 <h1 class="title">Our Products</h1>
25
26 <div class="products" id="product-list"></div>
27
28 <script src="js/script.js"></script>
29 </body>
30 </html>

```

Script.js

```

script.js > ...
const products = [
  { id: 1, name: "Laptop", price: 50000, desc: "Powerful laptop" },
  { id: 2, name: "Phone", price: 20000, desc: "Smart smartphone" },
  { id: 3, name: "Headphones", price: 3000, desc: "Noise cancelling" }
];

let cart = JSON.parse(localStorage.getItem("cart")) || [];

// LOAD PRODUCTS
if (document.getElementById("product-list")) {
  const container = document.getElementById("product-list");

  products.forEach(p => {
    container.innerHTML += `
      <div class="card">
        <h3>${p.name}</h3>
        <p>${p.desc}</p>
        <h4>₹${p.price}</h4>
        <button onclick="addToCart(${p.id})">Add to Cart</button>
      </div>
    `;
  });
}

// ADD
function addToCart(id) {
  let item = cart.find(c => c.id === id);

  if (item) item.qty++;
  else {
    let product = products.find(p => p.id === id);
    cart.push({ ...product, qty: 1 });
  }

  updateCart();
}

```

```

function updateCart() {
  localStorage.setItem("cart", JSON.stringify(cart));

  let count = cart.reduce((sum, i) => sum + i.qty, 0);
  let countEl = document.getElementById("cart-count");
  if (countEl) countEl.innerText = count;

  let cartItems = document.getElementById("cart-items");
  let total = 0;

  if (cartItems) {
    cartItems.innerHTML = "";

    cart.forEach(item => {
      total += item.price * item.qty;

      cartItems.innerHTML += `
        <div>
          ${item.name} x ${item.qty}
          <button onclick="changeQty(${item.id},1)">+</button>
          <button onclick="changeQty(${item.id},-1)">-</button>
          <button onclick="removeItem(${item.id})">✖</button>
        </div>
      `;
    });

    let totalEl = document.getElementById("total");
    if (totalEl) totalEl.innerText = total;
  }
}

// CHANGE QTY
function changeQty(id, change) {
  let item = cart.find(c => c.id === id);
  item.qty += change;
}

```

```

    }

    updateCart();
}

// REMOVE
function removeItem(id) {
    cart = cart.filter(c => c.id !== id);
    updateCart();
}

// ORDER
function placeOrder() {
    alert("Order placed successfully 🛒");
    cart = [];
    updateCart();
}

updateCart();

```

Style.css

```

body {
    font-family: 'Segoe UI';
    margin: 0;
    background: #f5f7fa;
}

/* NAVBAR */
nav {
    display: flex;
    justify-content: space-between;
    background: #1e272e;
    color: white;
    padding: 15px 30px;
}

nav a {
    color: white;
    margin-left: 15px;
    text-decoration: none;
}

/* TITLE */
.title {
    text-align: center;
    margin-top: 20px;
}

/* PRODUCT GRID */
.products {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 25px;
    padding: 30px;
}

/* CARD */
.card {

```

```

button:hover {
  background: #05c46b;
}

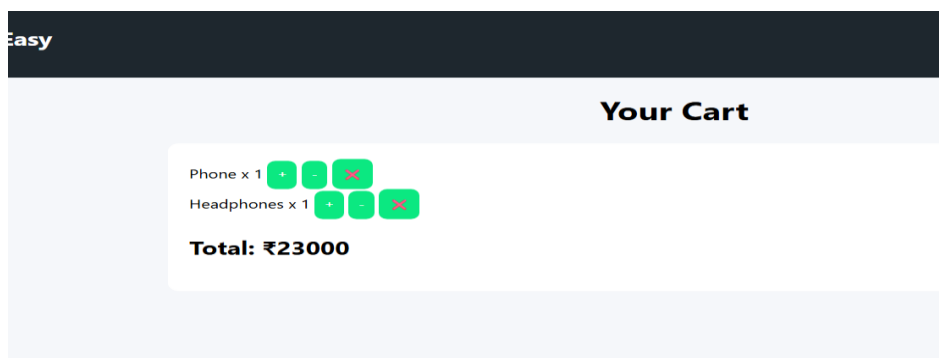
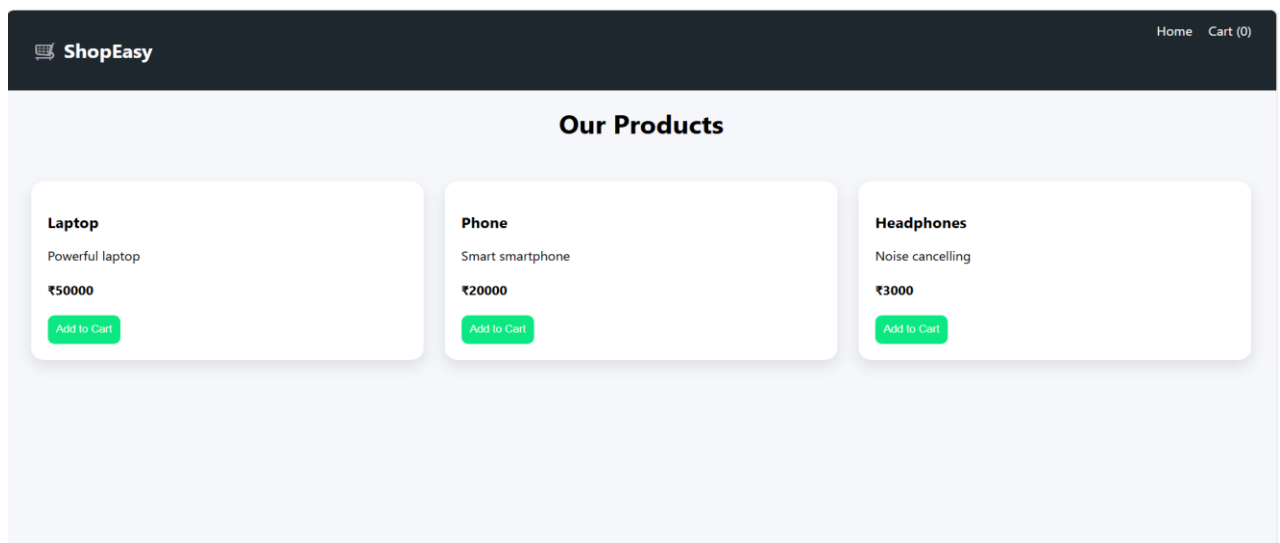
/* CART */
.cart-container {
  width: 60%;
  margin: auto;
  background: white;
  padding: 20px;
  border-radius: 10px;
}

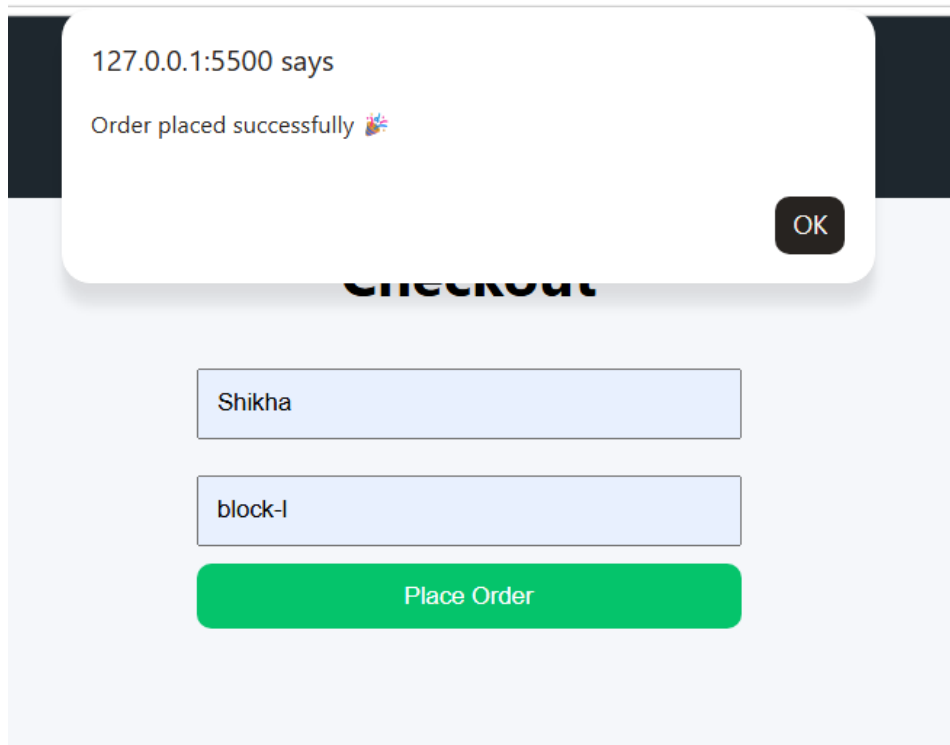
/* CHECKOUT */
.checkout-box {
  width: 300px;
  margin: auto;
  display: flex;
  flex-direction: column;
}

.checkout-box input {
  margin: 10px 0;
  padding: 10px;
}

```

OUTPUT





RESULT

A functional E-commerce website was successfully developed with product listing, cart management, and checkout simulation using HTML, CSS, and JavaScript.